Module 5 - Malware Analysis and Integrity Verification

5.1 - File Hashing and Integrity Verification

Objective

This section will give you a solid understanding of how to use Python to generate and verify file hashes, perform integrity checks, compare against known malware databases (such as VirusTotal or NSRL), and implement automation for routine malware analysis tasks using hashing techniques.

---

1. Introduction to Hashing in Security Context

Hashing is a one-way cryptographic function that converts input data (like a file) into a fixed-size string of characters, which acts like a digital fingerprint.

- **MD5** (Message Digest 5): Fast, but prone to collisions. Useful for quick indexing.
- **SHA-1**: Slightly more secure than MD5, still deprecated for cryptographic purposes.
- **SHA-256**: Stronger and used in most security applications today.

Hashing is commonly used to:

- Verify file integrity
- Detect tampering
- Identify malware (using known malicious hashes)
- Index samples for deduplication or cataloging

---

2. Calculating Hashes in Python

Python's built-in `hashlib` library provides robust support for MD5, SHA1, SHA256, etc.

Example: Hashing a file

```python
import hashlib

def calculate_hash(filepath, algorithm="sha256"):
    """
    Calculates the hash of a file using the specified algorithm.
    Supported algorithms: md5, sha1, sha256, sha512
    """
    hash_func = getattr(hashlib, algorithm)()

    with open(filepath, "rb") as f:
        while chunk := f.read(4096):  # Read file in 4KB chunks for large files
            hash_func.update(chunk)

    return hash_func.hexdigest()

# Example usage
file_path = "sample_malware.exe"
print("SHA-256:", calculate_hash(file_path, "sha256"))
print("MD5:", calculate_hash(file_path, "md5"))
```

Code Notes:

- `getattr(hashlib, algorithm)` dynamically selects the hashing algorithm.
- Reading in chunks is important for memory efficiency when handling large binaries.
- `hexdigest()` returns the readable string form of the hash.

---

3. Verifying File Integrity

Suppose you download a file with a published hash. You can compare your computed hash to verify it.

```
expected_sha256 = "de9f2c7fd25e1b3afad3e85a0bd17d9bdf5f1e931a9aa06f89aef61c2730d8b6"
file_path = "payload.exe"

actual_sha256 = calculate_hash(file_path, "sha256")

if actual_sha256.lower() == expected_sha256.lower():
    print("[+] File integrity verified.")
else:
    print("[-] WARNING: File integrity check failed!")
```

This can be automated in workflows to validate tools, payloads, or sample archives.

---

4. Cross-Referencing with Malware Databases

Option 1: VirusTotal API

You can query VirusTotal for known hashes.

```
import requests

def check_virustotal(hash_value, api_key):
    url = f"https://www.virustotal.com/api/v3/files/{hash_value}"
    headers = {"x-apikey": api_key}

    response = requests.get(url, headers=headers)
    if response.status_code == 200:
```

```
        data = response.json()
        detections = data["data"]["attributes"]["last_analysis_stats"]
        print(f"Detections: {detections['malicious']}/{sum(detections.values())}")
    else:
        print("[-] Hash not found or error querying VirusTotal.")

# Usage
vt_api_key = "YOUR_API_KEY_HERE"
check_virustotal(actual_sha256, vt_api_key)
```

> This allows you to verify whether the file has been submitted before and if it's known to AV vendors.

---

5. Hashing Folders Recursively

You can generate a hash inventory of an entire directory—useful when analyzing batches of files.

```
import os

def hash_directory(directory, algorithm="sha256"):
    hashes = {}
    for root, _, files in os.walk(directory):
        for file in files:
            filepath = os.path.join(root, file)
            try:
                file_hash = calculate_hash(filepath, algorithm)
                hashes[filepath] = file_hash
            except Exception as e:
                print(f"Error hashing {filepath}: {e}")
    return hashes

# Example
```

```
hashes = hash_directory("malware_samples/")
for path, digest in hashes.items():
    print(f"{digest}  {path}")
```

---

6. Using Hashes as Malware Fingerprints

Malware samples may be indexed in databases by their hash:

Sample Name	SHA256	Family
agentTesla1.exe	f5a3...	AgentTesla
trojan.doc	e4d8...	Emotet

Python can automate checks across such datasets.

---

7. Logging and Reporting Hashes

Hashes can be saved for later comparison or reporting:

```
import json

def save_hash_report(hashes, output="hashes.json"):
    with open(output, "w") as f:
        json.dump(hashes, f, indent=4)

# Example usage
save_hash_report(hashes)
```

Or export to CSV:

```
import csv

def save_to_csv(hashes, output="hashes.csv"):
    with open(output, "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["File Path", "Hash"])
        for path, digest in hashes.items():
            writer.writerow([path, digest])
```

---

8. Hash Collision Considerations

While MD5 and SHA1 are still widely used in threat intelligence, they are **not collision-resistant**:

- **Attackers** can create two different files with the same MD5 hash (MD5 collision attacks).
- Always prefer **SHA-256** or better for security-sensitive operations.

---

9. Practical Use Case: Malware Sample Tracker

Build a lightweight tool that:

- Takes a folder of files
- Hashes each sample
- Checks them against VirusTotal
- Generates a report of known/unknown samples

This is a common automation pattern in malware triage environments.

10. Exercise (For the Student)

Task: Build a Python script that:

1. Accepts a file or folder as input.
2. Calculates MD5, SHA1, and SHA256 for each file.
3. Checks SHA256 hash in VirusTotal (via API).
4. Outputs a formatted report in JSON or CSV.
5. Includes status: `{"known_malware": True, "detections": 32}`

5.2 - Basic Static File Analysis: Headers, Magic Bytes, and Strings

Objective

This section introduces essential static file analysis techniques using Python. These include analyzing file headers, identifying file types using magic bytes, and extracting readable strings from binaries. These techniques are foundational in malware analysis, incident response, and forensic investigations.

---

1. Introduction to Static File Analysis

Static file analysis refers to examining a file **without executing it**, allowing analysts to safely understand file characteristics, origin, structure, and potential malicious behavior.

Core techniques include:

- Inspecting **file headers** (e.g., PE, ELF, PDF)
- Detecting **magic numbers** to confirm file type
- Extracting **ASCII/Unicode strings** for clues (e.g., URLs, commands, hostnames)

This type of analysis is fast, safe, and effective for triage in security workflows.

---

2. Reading Raw Binary Data

Before analyzing content, you must read the file safely in binary mode.

```
def read_binary_file(filepath):
    with open(filepath, "rb") as f:
        return f.read()

binary_data = read_binary_file("sample_malware.exe")
print(f"File size: {len(binary_data)} bytes")
```

This raw data becomes the basis for all following analysis steps.

---

3. Analyzing File Headers

Many file formats include fixed header bytes to identify the file type.

Common Headers:

Format	Magic Bytes (Hex)	ASCII
ELF (Linux executable)	0x7F 45 4C 46	.ELF
PE (Windows EXE/DLL)	0x4D 5A	MZ
PDF	0x25 50 44 46	%PDF

Example: Detecting ELF or PE

```
def detect_file_type(data):
    if data.startswith(b"\x7fELF"):
        return "ELF Executable (Linux)"
    elif data.startswith(b"MZ"):
        return "PE Executable (Windows)"
    elif data.startswith(b"%PDF"):
        return "PDF Document"
    else:
        return "Unknown format"

print("Detected Type:", detect_file_type(binary_data))
```

This approach helps quickly identify malicious executables or documents.

---

4. Extracting Magic Bytes for Generic Detection

To further support unknown or rare file types, use a magic byte database (e.g., `python-magic`).

```
import magic

def guess_file_type(filepath):
    file_type = magic.from_file(filepath)
    print("File type:", file_type)

guess_file_type("sample_malware.exe")
```

The `python-magic` library mimics the `file` command on Unix systems and can identify complex formats.

5. Extracting Strings from Binaries

Malware often contains hardcoded domains, IPs, commands, registry keys, or error messages in plain text. These can be extracted using ASCII or Unicode string patterns.

Extracting Printable ASCII Strings

```
import re

def extract_ascii_strings(data, min_length=4):
    pattern = rb"[ -~]{%d,}" % min_length  # ASCII range from space to ~
    return [s.decode("ascii") for s in re.findall(pattern, data)]

strings = extract_ascii_strings(binary_data)
for s in strings[:10]:  # Show first 10
    print(s)
```

Extracting Unicode Strings (UTF-16 LE)

```
def extract_unicode_strings(data, min_length=4):
    pattern = (rb"(?:[\x20-\x7E]\x00){%d,}" % min_length)  # UTF-16LE
    return [s.decode("utf-16le") for s in re.findall(pattern, data)]

unicode_strings = extract_unicode_strings(binary_data)
```

These string dumps are extremely useful for identifying command-and-control URLs, mutexes, or payload hints.

---

6. Identifying Embedded Executables or Scripts

Advanced malware might contain **embedded files** such as:

- JavaScript (JS)
- PowerShell
- Base64 blobs
- Second-stage EXEs or DLLs

You can look for indicators like:

```
suspicious_patterns = [
    "powershell",
    "cmd.exe",
    "vbs",
    "wget",
    "base64",
    "FromBase64String",
    "CreateObject",
    "VirtualAlloc",
    "rundll32",
    "http",
    ".onion"
]

for line in strings:
    if any(p in line.lower() for p in suspicious_patterns):
        print("[SUSPICIOUS] " + line)
```

This technique can help flag obfuscated payloads or downloader behaviors.

---

7. File Entropy (Optional Advanced Topic)

High entropy often indicates compressed or encrypted data (e.g., packers or embedded payloads). Calculating Shannon entropy:

```
import math
from collections import Counter

def calculate_entropy(data):
    if not data:
        return 0
    byte_counts = Counter(data)
    total_bytes = len(data)
    entropy = -sum((count / total_bytes) * math.log2(count / total_bytes) for count in byte_counts.values())
    return entropy

print("File Entropy:", calculate_entropy(binary_data))
```

- Entropy ~7.5–8.0 = highly packed/encrypted
- Entropy ~4–6 = normal binaries
- Entropy < 3 = likely plain text

---

8. Summary Workflow for Static Analysis

1. Load file in binary mode
2. Check header/magic bytes
3. Use `magic` to detect deeper types
4. Extract ASCII and Unicode strings
5. Search for suspicious keywords
6. Optionally, calculate entropy
7. Export indicators

9. Example: Complete Static Analysis Script

```python
def static_analysis(filepath):
    data = read_binary_file(filepath)

    print("[+] File type:", detect_file_type(data))
    print("[+] Magic:", magic.from_file(filepath))

    print("\n[+] Extracted ASCII Strings (Top 10):")
    for s in extract_ascii_strings(data)[:10]:
        print("-", s)

    print("\n[+] Suspicious Strings:")
    for s in extract_ascii_strings(data):
        if any(x in s.lower() for x in suspicious_patterns):
            print(">>", s)

    print("\n[+] Entropy:", calculate_entropy(data))

# Run on a sample file
static_analysis("sample_payload.bin")
```

10. Use Cases in Cybersecurity

- Triaging unknown attachments or downloads
- Detecting embedded or obfuscated code
- Identifying malware families by strings (e.g., Cobalt Strike beacon configs)
- Creating YARA rules based on extracted strings or headers

11. Exercise

Build a Python script that:

- Accepts a file path from the user
- Detects file format (header + magic)
- Extracts and saves ASCII + Unicode strings to a `.txt`
- Identifies suspicious indicators
- Calculates entropy
- Saves the report in JSON format

5.3 - IOC Extraction from Scripts and Binaries

Objective

IOC (Indicator of Compromise) extraction is a fundamental technique in malware analysis and threat hunting. This topic explores how to extract key indicators such as **IP addresses**, **domain names**, **file hashes**, **URLs**, and **email addresses** from potentially malicious **scripts** (e.g., `.js`, `.vbs`, `.ps1`) and **binary files** (e.g., `.exe`, `.dll`).

Python enables automation of this process, making it scalable for use in sandboxes, detection pipelines, and incident response.

---

1. What are IOCs?

Indicators of Compromise (IOCs) are data artifacts observed on a network or in a system that indicate a possible intrusion or malicious activity.

Typical IOCs include:

- IP addresses

- Domains
- URLs
- Email addresses
- File hashes (MD5, SHA1, SHA256)
- Filenames or mutex names

---

2. Extracting IOCs from Text-based Scripts

Malicious scripts often include hardcoded infrastructure data, command-and-control URLs, download links, etc. Extracting IOCs from plaintext is relatively straightforward.

Example: IOC Extraction from PowerShell or JavaScript Files

```
import re

def extract_iocs_from_text(content):
    iocs = {}

    # Regex patterns
    ip_pattern = r"\b(?:[0-9]{1,3}\.){3}[0-9]{1,3}\b"
    domain_pattern = r"\b(?:[a-zA-Z0-9-]+\.)+[a-zA-Z]{2,}\b"
    url_pattern = r"https?://[^\s'\"<>]+"
    email_pattern = r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}"
    hash_pattern = r"\b[a-fA-F0-9]{32,64}\b"

    # Find matches
    iocs['ips'] = re.findall(ip_pattern, content)
    iocs['domains'] = re.findall(domain_pattern, content)
    iocs['urls'] = re.findall(url_pattern, content)
    iocs['emails'] = re.findall(email_pattern, content)
```

```
    iocs['hashes'] = re.findall(hash_pattern, content)

    return iocs
```

Usage Example

```
with open("malicious_script.ps1", "r", encoding="utf-8", errors="ignore") as f:
    content = f.read()

iocs = extract_iocs_from_text(content)

print("[+] Extracted IOCs:")
for k, v in iocs.items():
    print(f"{k}: {set(v)}")
```

> Tip: `set()` is used to remove duplicates.

---

3. Extracting IOCs from Binary Files

To extract IOCs from binaries, the common approach is:

1. Read the file as binary
2. Extract printable strings
3. Apply IOC regexes on the strings

Extracting Strings from Binary Files

```
def extract_strings_from_binary(binary_data, min_length=4):
    pattern = rb"[ -~]{%d,}" % min_length  # ASCII printable characters
```

```
    return [s.decode("ascii", errors="ignore") for s in re.findall(pattern, binary_data)]
```

Apply IOC Regex to Strings

```
def extract_iocs_from_binary(filepath):
    with open(filepath, "rb") as f:
        data = f.read()

    strings = extract_strings_from_binary(data)
    combined = "\n".join(strings)

    return extract_iocs_from_text(combined)
```

---

4. Real-World Patterns to Look For

Some malware uses non-obvious indicators:

- **Encoded URLs** (Base64, hex)
- **Split strings** (e.g., `"http" + "://malicious.com"`)
- **Obfuscation via character arrays or concatenation**

Simple Base64 IOC Decoding

```
import base64

def find_base64_iocs(content):
    base64_pattern = r"(?:[A-Za-z0-9+/]{10,}={0,2})"
    matches = re.findall(base64_pattern, content)
```

```
    decoded = []
    for b64 in matches:
        try:
            out = base64.b64decode(b64).decode("utf-8")
            if "http" in out or "." in out:
                decoded.append(out)
        except:
            continue
    return decoded
```

This logic attempts to decode suspicious-looking Base64 strings and extract possible indicators hidden in them.

---

5. Exporting IOCs

You can export IOCs to multiple formats (CSV, JSON, plain text) for triage or SIEM ingestion.

Example: Save IOCs to JSON

```
import json

def save_iocs_json(iocs, output_file):
    with open(output_file, "w") as f:
        json.dump(iocs, f, indent=2)
```

---

6. Complete IOC Extraction Script

```
def extract_iocs(filepath):
    if filepath.endswith((".ps1", ".js", ".vbs", ".txt")):
```

```
        with open(filepath, "r", encoding="utf-8", errors="ignore") as f:
            content = f.read()
        iocs = extract_iocs_from_text(content)

    else:
        iocs = extract_iocs_from_binary(filepath)

    # Optionally decode Base64 IOCs
    decoded = find_base64_iocs(content if 'content' in locals() else "")
    if decoded:
        iocs['decoded_base64'] = decoded

    return iocs

# Run the tool
path = "sample_file.bin"
results = extract_iocs(path)
save_iocs_json(results, "output_iocs.json")
```

---

7. Threat Intelligence Integration

Extracted IOCs can be:

- Sent to a Threat Intelligence Platform (MISP, OTX)
- Used in alerting rules (YARA, Sigma, Snort)
- Fed into IOC enrichment services (VirusTotal, AbuseIPDB)

---

8. Bonus: Creating a Simple IOC Scanner

You can convert extracted IOCs into a scanner to apply against logs or memory dumps:

```python
def scan_file_for_iocs(ioc_dict, target_file):
    with open(target_file, "r", encoding="utf-8", errors="ignore") as f:
        content = f.read()

    for ioc_type, iocs in ioc_dict.items():
        for indicator in iocs:
            if indicator in content:
                print(f"[MATCH] {ioc_type}: {indicator}")
```

---

9. Exercise

1. Write a Python tool that accepts any file (script or binary).
2. Extract all relevant IOCs (URLs, IPs, domains, hashes).
3. Include base64 decoding if applicable.
4. Save the results to both `.json` and `.csv`.
5. Bonus: Allow scanning another file for matches using those IOCs.

5.4 - Behavioral Analysis Integration with Python Sandbox Emulation

Objective

The goal of this topic is to demonstrate how Python can be used to build or interface with sandbox-like environments to analyze the **behavior** of potentially malicious executables or scripts in a **controlled, emulated** environment. This approach captures **runtime behavior** such as:

- File system modifications
- Registry access

- Network connections
- Process creation
- API calls

Behavioral analysis complements static techniques by revealing real intent, evasion mechanisms, and persistence methods not evident in code alone.

---

1. What Is Behavioral Analysis?

Behavioral analysis observes **how a sample behaves during execution**. It typically involves:

- Executing malware in an isolated environment
- Monitoring system-level interactions
- Logging events for further investigation

Why use Python?

Python can:

- Interface with Windows APIs using `ctypes` or `pywin32`
- Emulate limited execution using `unicorn` or `Qiling`
- Monitor system calls and file activity
- Serve as glue between Cuckoo, PE executors, and your detection logic

---

2. Building a Mini Python Sandbox with `subprocess`, File Watching, and Hooks

This section demonstrates a basic behavioral sandbox concept for **Windows executables**.

Step 1: Setup a Temporary Environment

```python
import os
import shutil
import tempfile

# Create an isolated temporary directory to simulate environment
sandbox_dir = tempfile.mkdtemp()
print(f"[+] Sandbox initialized at {sandbox_dir}")

# Copy sample to sandbox
sample_path = "malicious.exe"
sandbox_sample = os.path.join(sandbox_dir, "sample.exe")
shutil.copy(sample_path, sandbox_sample)
```

Step 2: Monitor File System Changes (Before & After Execution)

```python
def list_files_recursive(root_dir):
    file_map = {}
    for root, _, files in os.walk(root_dir):
        for file in files:
            path = os.path.join(root, file)
            try:
                file_map[path] = os.stat(path).st_size
            except:
                continue
    return file_map
```

Step 3: Execute the Sample in Controlled Mode

```
import subprocess
import time

# Snapshot before execution
before = list_files_recursive(sandbox_dir)

# Run the sample (assumes Windows sandbox; change if Linux)
print("[*] Executing sample...")
try:
    subprocess.run([sandbox_sample], cwd=sandbox_dir, timeout=10)
except subprocess.TimeoutExpired:
    print("[!] Sample timed out")

# Snapshot after execution
after = list_files_recursive(sandbox_dir)
```

Step 4: Detect Created or Modified Files

```
def detect_changes(before, after):
    created = [f for f in after if f not in before]
    modified = [f for f in after if f in before and after[f] != before[f]]
    return created, modified

created, modified = detect_changes(before, after)
print(f"[+] Created files: {created}")
print(f"[+] Modified files: {modified}")
```

---

3. Monitoring Process and Network Activity

You can use tools like `psutil` and `scapy` (or even raw sockets on Linux) to detect child processes and connections.

Example: List All New Processes Launched

```
import psutil

def capture_processes():
    return {p.pid: p.name() for p in psutil.process_iter(['name'])}

# Before
before_procs = capture_processes()

# Execute Sample
subprocess.run([sandbox_sample], cwd=sandbox_dir, timeout=10)

# After
after_procs = capture_processes()

# Diff
new_pids = set(after_procs) - set(before_procs)
print("[+] New processes detected:")
for pid in new_pids:
    print(f"PID {pid} - {after_procs[pid]}")
```

---

4. Optional: Network Monitoring with `scapy` or `pydivert`

For more advanced cases:

- Use `scapy` or `tcpdump` to sniff connections

- On Windows, use `pydivert` to hook outbound/inbound traffic

> ⚠ Note: Requires admin/root and isolation.

---

5. Emulating Behavior with Unicorn (Advanced Use Case)

You can use Unicorn Engine to **emulate shellcode or x86 execution** in a safe memory region.

Example: Executing a Shellcode (Basic)

```
from unicorn import Uc, UC_ARCH_X86, UC_MODE_32
from unicorn.x86_const import *

# NOP + INT3 as placeholder
shellcode = b"\x90\xCC"

# Setup memory and emulator
ADDRESS = 0x1000000
emu = Uc(UC_ARCH_X86, UC_MODE_32)
emu.mem_map(ADDRESS, 2 * 1024 * 1024)
emu.mem_write(ADDRESS, shellcode)

try:
    emu.emu_start(ADDRESS, ADDRESS + len(shellcode))
except Exception as e:
    print(f"[!] Exception during emulation: {e}")
```

You can hook memory, syscalls, or API emulation as needed.

---

6. Logging Results for Analysis

Collected data should be exported for triage or dashboards.

Example: Export to JSON

```
import json

sandbox_log = {
    "created_files": created,
    "modified_files": modified,
    "new_processes": list(new_pids)
}

with open("sandbox_report.json", "w") as f:
    json.dump(sandbox_log, f, indent=2)

print("[+] Behavior report saved to sandbox_report.json")
```

---

7. Integration with Existing Sandboxes

Instead of building your own, you can integrate Python with:

- **Cuckoo Sandbox** (deprecated, still useful for local use)
- **CAPEv2** (Cuckoo fork with better malware unpacking)
- **Any.run API**, **JoeSandbox API**, etc.

Python scripts can automate:

- Upload of samples

- Collection of behavior logs
- Triggering analysis and fetching results

---

8. Exercise

Create a behavioral sandbox script that:

1. Accepts a `.exe` or `.dll` sample
2. Runs it in an isolated folder
3. Monitors filesystem and process changes
4. Records the results in a structured JSON report
5. Bonus: Detects if persistence mechanisms (e.g., file in Startup folder) are created

5.5 - Integrating YARA for Signature-Based Detection

Objective

In this topic, we explore how to integrate the **YARA engine** into Python scripts for **malware identification** and **threat detection** based on predefined **binary/text patterns** (rules). YARA is the industry standard for **signature-based detection** of malware, often referred to as the "pattern-matching Swiss Army knife."

---

1. What is YARA?

YARA allows you to create **rules** that describe patterns or characteristics of malware or suspicious files. These rules are evaluated against files or memory regions to detect threats.

Example YARA Rule:

```
rule Suspicious_PDF_Dropper
{
    strings:
        $a = "%PDF"
        $b = "obj"
        $c = "stream"

    condition:
        all of them
}
```

This rule matches any PDF file that contains all the specified strings. Rules can also detect specific **hex patterns**, **wildcards**, **file sizes**, or **complex logic conditions**.

---

2. Installing and Using YARA with Python

To use YARA in Python, install the `yara-python` module:

```
pip install yara-python
```

Import it in your script:

```
import yara
```

---

3. Writing and Compiling YARA Rules in Python

You can load YARA rules from strings or files.

From a File:

```
rules = yara.compile(filepath='malware_rules.yar')
```

From a String:

```
rule_str = """
rule HelloRule {
    strings:
        $hello = "Hello World"
    condition:
        $hello
}
"""
rules = yara.compile(source=rule_str)
```

---

4. Scanning Files with YARA

After compilation, use the `match()` function to scan files:

```
matches = rules.match("suspicious_file.exe")
for match in matches:
    print(f"[+] Match: {match.rule}")
```

You can also scan **memory buffers**:

```
with open("suspicious_file.exe", "rb") as f:
    data = f.read()
```

```
matches = rules.match(data=data)
```

---

5. Working with Metadata and Tags

YARA rules support **tags** and **metadata**, which are useful for reporting and triage.

```
rule Trojan_Foo : trojan keylogger
{
    meta:
        author = "analyst"
        threat_level = 3
        description = "Detects Trojan Foo behavior"

    strings:
        $s1 = "malicious_string"

    condition:
        $s1
}
```

In Python:

```
for match in matches:
    print(f"Rule: {match.rule}")
    print("Tags:", match.tags)
    print("Metadata:", match.meta)
```

---

6. Organizing Rule Repositories

For large projects, organize rules by families:

```
rules/
├── pdf/
│   └── dropper_pdf.yar
├── trojans/
│   └── generic_trojans.yar
├── ransomware/
│   └── known_variants.yar
```

Then compile all with:

```
rules = yara.compile(filepaths={
    'pdf': 'rules/pdf/dropper_pdf.yar',
    'trojans': 'rules/trojans/generic_trojans.yar'
})
```

---

7. Error Handling and Debugging Rules

YARA will raise `SyntaxError` or `YaraSyntaxError` if the rules are invalid.

```
try:
    rules = yara.compile(filepath="broken_rule.yar")
except yara.SyntaxError as e:
    print(f"[!] YARA syntax error: {e}")
```

---

8. Real-World Example: IOC Scanner

This scanner loads rules and checks if a file matches any known malware patterns.

```
import yara
import sys
import os

def scan_file(file_path, rule_file):
    rules = yara.compile(filepath=rule_file)
    matches = rules.match(file_path)

    if matches:
        print(f"[!] Suspicious: {file_path}")
        for match in matches:
            print(f" -> Rule matched: {match.rule} | Tags: {match.tags} | Meta: {match.meta}")
    else:
        print(f"[+] Clean: {file_path}")

if __name__ == "__main__":
    scan_file("unknown.exe", "rules/all_rules.yar")
```

---

9. Integration with File Upload Pipeline

In threat hunting or email gateway scenarios, YARA can be used to **scan uploaded files**:

```
def handle_uploaded_file(file_bytes):
    rules = yara.compile(filepath='rules/enterprise.yar')
    return rules.match(data=file_bytes)
```

10. Use Case: Scan Directory Recursively

```
import os

def scan_directory(dir_path, rules):
    for root, _, files in os.walk(dir_path):
        for file in files:
            try:
                full_path = os.path.join(root, file)
                matches = rules.match(full_path)
                if matches:
                    print(f"[+] Match in {file}: {[m.rule for m in matches]}")
            except Exception as e:
                print(f"Error scanning {file}: {e}")
```

11. Exporting Match Results to JSON

```
import json

results = []

for match in matches:
    result = {
        "rule": match.rule,
        "tags": match.tags,
        "meta": match.meta
    }
    results.append(result)
```

```
with open("yara_results.json", "w") as f:
    json.dump(results, f, indent=2)
```

Summary

Feature	Support in YARA
String/Hex Pattern Matching	✅
Logic + Conditions	✅
File + Memory Scanning	✅
Python Integration	✅ yara-python
Metadata + Tags	✅
File Types/Size/Hash Checks	✅

Exercise

1. Create a YARA rule to detect base64-encoded PowerShell in scripts.
2. Compile and scan all `.ps1` files in a given directory.
3. Export any matches to `base64_powershell_alerts.json`.

5.6 - Extracting Suspicious Code Blocks from Binaries

Objective

This topic focuses on using Python to extract **suspicious binary code segments** from executables and other binary files. The aim is to isolate **potentially malicious payloads**, such as embedded shellcode, encrypted blobs, or obfuscated routines, for deeper analysis. This process is essential in **malware triage**, **sandboxing**, **reverse engineering**, and **threat hunting workflows**.

We will cover:

- Reading and parsing binary files
- Locating common shellcode patterns (NOP sleds, syscall stubs)
- Detecting suspicious entropy regions
- Using pattern-based signatures (regex, magic bytes)
- Carving out code blocks and saving them for analysis

---

1. Understanding Binary Code Structures

Binary files (EXE, DLL, ELF, etc.) are composed of **headers**, **sections**, and **data/code segments**. Suspicious code is often embedded in:

- `.text` (code)
- `.data` or `.rdata` (obfuscated payloads)
- Appended to the end of the file
- Packed inside a single section using custom encryption

Tools like `PEfile` and `lief` allow us to parse structure and identify anomalous areas.

---

2. Reading and Dumping Raw Bytes from Files

```
def read_binary_file(file_path):
    with open(file_path, "rb") as f:
```

```
        data = f.read()
    return data

binary_data = read_binary_file("sample_malware.exe")
print(f"Read {len(binary_data)} bytes from file.")
```

---

3. Identifying Shellcode Patterns (NOP Sleds, Syscalls)

A common tactic is to look for **NOP sleds** (repeating `0x90`) or syscall markers.

```
import re

# NOP sled detection (e.g., 20+ NOPs in a row)
pattern = b"\x90{20,}"

matches = list(re.finditer(pattern, binary_data))
for m in matches:
    start, end = m.span()
    print(f"Suspicious NOP sled from offset {start} to {end}")
```

---

4. Detecting High Entropy Regions (Potential Encryption)

High entropy often indicates packed or encrypted data.

```
import math

def shannon_entropy(data):
    if not data:
```

```
        return 0
    freq = {b: data.count(b) for b in set(data)}
    total = len(data)
    return -sum((count / total) * math.log2(count / total) for count in freq.values())

# Sliding window entropy scan
window_size = 256
for i in range(0, len(binary_data) - window_size, 128):
    chunk = binary_data[i:i + window_size]
    entropy = shannon_entropy(chunk)
    if entropy > 7.5:
        print(f"High entropy detected at offset {i} (Entropy: {entropy:.2f})")
```

---

5. Extracting Specific Code Blocks from Matched Patterns

Once a region is marked as suspicious, we extract a block for analysis:

```
def extract_block(data, start, length=512):
    block = data[start:start+length]
    with open(f"suspect_block_{start}.bin", "wb") as f:
        f.write(block)
    print(f"Extracted block of {len(block)} bytes at offset {start}")
```

Example usage after NOP match:

```
for m in matches:
    start = m.start()
    extract_block(binary_data, start)
```

---

6. Using Regular Expressions for Heuristic Patterns

You can also match encoded payloads (e.g., base64 in binaries):

```
base64_pattern = rb"[A-Za-z0-9+/]{100,}={0,2}"
for m in re.finditer(base64_pattern, binary_data):
    print(f"Potential base64 blob at {m.start()}")
```

---

7. Optional: PE Header Analysis with `pefile`

To focus on `.text` or suspicious sections:

```
import pefile

pe = pefile.PE("sample_malware.exe")

for section in pe.sections:
    name = section.Name.strip(b'\x00').decode()
    entropy = section.get_entropy()
    print(f"Section {name} | Entropy: {entropy:.2f}")
    if entropy > 7.5:
        offset = section.PointerToRawData
        size = section.SizeOfRawData
        suspicious = binary_data[offset:offset+size]
        with open(f"suspicious_section_{name}.bin", "wb") as f:
            f.write(suspicious)
```

This allows carving based on **section entropy**.

---

8. Carving Embedded Executables or Shellcode

Some malware drops or embeds entire PE files:

```
# PE magic bytes: 'MZ' or 0x4D 0x5A
mz_pattern = b"\x4D\x5A"

offsets = [m.start() for m in re.finditer(mz_pattern, binary_data)]
for offset in offsets:
    extract_block(binary_data, offset, 2048)
```

---

9. Saving Results and Creating Reports

You can automatically label and store the findings:

```
import json

results = []
for match in matches:
    results.append({
        "offset": match.start(),
        "length": match.end() - match.start(),
        "pattern": "NOP sled"
    })

with open("suspicious_blocks.json", "w") as f:
    json.dump(results, f, indent=2)
```

---

10. Practical Use Case: Automated Malware Triage Script

```python
def analyze_file(file_path):
    data = read_binary_file(file_path)
    print(f"Analyzing {file_path}...")

    nop_hits = list(re.finditer(b"\x90{16,}", data))
    print(f"Detected {len(nop_hits)} NOP sleds")

    for i, match in enumerate(nop_hits):
        extract_block(data, match.start(), 512)

    print("Entropy scan starting...")
    for i in range(0, len(data) - 256, 128):
        entropy = shannon_entropy(data[i:i+256])
        if entropy > 7.5:
            extract_block(data, i, 512)

# Run the triage
analyze_file("unknown_payload.bin")
```

Summary Table

Technique	Purpose
NOP sled pattern	Detect common shellcode preambles
High entropy scan	Locate packed or encrypted blobs
Magic byte signature search	Extract embedded PE or ELF files
Shannon entropy + sliding win	Generic heuristic for suspicious code

Technique	Purpose
PE header analysis	Carve out malicious sections
Regex for encoded blobs	Find base64 and other obfuscations

Exercise

1. Download a malware sample (or benign executable) from a trusted test dataset.
2. Write a Python script that:
   - Parses the binary file
   - Scans for NOP sleds
   - Computes section entropies using `pefile`
   - Extracts and saves any suspicious code blocks
3. Save results as both binary dumps and a JSON report.

5.7 - Memory Dump Carving with Python

Objective

This topic covers how to programmatically analyze and extract suspicious data from memory dumps (RAM captures) using Python. Memory dumps are rich with valuable forensic information such as:

- Injected code and shellcode
- Credentials in plaintext
- Malware strings and configuration
- Embedded PE files or scripts
- IOCs (IP addresses, domains, file paths)

Python enables lightweight, targeted memory carving without needing a full-blown memory analysis framework like Volatility.

---

1. Reading Raw Memory Dumps

Memory dumps are binary files, usually several megabytes or gigabytes in size. First, we read the file in chunks to avoid high memory usage.

```
def read_memory_in_chunks(file_path, chunk_size=1024 * 1024):  # 1 MB chunks
    with open(file_path, 'rb') as f:
        while True:
            chunk = f.read(chunk_size)
            if not chunk:
                break
            yield chunk
```

This function allows efficient processing of large files without loading the entire dump into memory at once.

---

2. Searching for Embedded Executables (Magic Bytes)

Many malware samples inject or drop entire PE executables into memory. We search for the "MZ" magic byte that starts all Windows PE files.

```
import re

def find_pe_headers(file_path):
    with open(file_path, "rb") as f:
        data = f.read()
```

```
    # MZ header (0x4D 0x5A) — typical of Windows executables
    pattern = b"\x4D\x5A"
    offsets = [match.start() for match in re.finditer(pattern, data)]

    for i, offset in enumerate(offsets):
        print(f"[+] Possible PE file found at offset {offset}")
        # Save a carved portion (2 KB) for further analysis
        with open(f"carved_pe_{i}.bin", "wb") as f_out:
            f_out.write(data[offset:offset+2048])
```

This method is ideal for quickly detecting injected PEs or unpacked malware payloads.

---

3. Detecting Shellcode Patterns (NOP Sleds, Syscalls)

Shellcode often uses a sequence of NOPs (`0x90`) followed by encoded or syscall instructions.

```
def detect_shellcode_regions(data):
    # Look for sequences of 16 or more NOPs (0x90)
    pattern = b"\x90{16,}"
    return [(match.start(), match.end()) for match in re.finditer(pattern, data)]

# Example usage
with open("memdump.raw", "rb") as f:
    data = f.read()
    regions = detect_shellcode_regions(data)
    for i, (start, end) in enumerate(regions):
        print(f"[!] NOP sled detected at {start}")
        with open(f"shellcode_region_{i}.bin", "wb") as f_out:
            f_out.write(data[start:start+512])  # Save 512 bytes from match
```

This is particularly useful when shellcode was injected into process memory using techniques like reflective DLL injection or `VirtualAlloc`.

---

4. Extracting IOCs from Memory (Strings, IPs, Domains)

Many IOCs such as command-and-control (C2) domains or plaintext strings can be found in memory.

```
import re

def extract_ascii_strings(data, min_length=4):
    pattern = rb"[ -~]{%d,}" % min_length  # Printable ASCII
    return [match.group().decode('ascii', errors='ignore') for match in re.finditer(pattern, data)]

# Extract strings from memory dump
with open("memdump.raw", "rb") as f:
    data = f.read()

strings = extract_ascii_strings(data)
print(f"Extracted {len(strings)} strings")

# Search for suspicious indicators
ioc_indicators = []
for s in strings:
    if re.match(r"\b\d{1,3}(\.\d{1,3}){3}\b", s):  # IP address
        ioc_indicators.append(("IP", s))
    elif ".onion" in s or s.endswith(".xyz") or s.endswith(".ru"):
        ioc_indicators.append(("Domain", s))
    elif "password" in s.lower() or "admin" in s.lower():
        ioc_indicators.append(("Keyword", s))

# Save found IOCs
```

```
with open("memory_iocs.txt", "w") as f_out:
    for type_, val in ioc_indicators:
        f_out.write(f"{type_}: {val}\n")
```

This technique helps in triage and threat hunting by quickly identifying likely malicious artifacts.

---

5. High Entropy Scanning in Memory

High entropy blocks often suggest encrypted or packed data, commonly used by malware to evade detection.

```
import math

def shannon_entropy(data):
    if not data:
        return 0
    entropy = 0
    for b in set(data):
        p = data.count(b) / len(data)
        entropy -= p * math.log2(p)
    return entropy

def scan_entropy_regions(data, window=512, threshold=7.5):
    high_entropy_blocks = []
    for i in range(0, len(data) - window, 256):
        chunk = data[i:i+window]
        entropy = shannon_entropy(chunk)
        if entropy > threshold:
            high_entropy_blocks.append((i, entropy))
    return high_entropy_blocks

# Run entropy scan
```

```
blocks = scan_entropy_regions(data)
for offset, entropy in blocks:
    print(f"High entropy block at offset {offset}, entropy={entropy:.2f}")
```

These blocks may contain obfuscated configurations or encrypted payloads.

---

6. Putting It All Together – Memory Carving Framework

```
def memory_carving_pipeline(file_path):
    print(f"Analyzing memory dump: {file_path}")
    with open(file_path, "rb") as f:
        data = f.read()

    # Step 1: Carve PE files
    pe_hits = [m.start() for m in re.finditer(b"\x4D\x5A", data)]
    for i, offset in enumerate(pe_hits):
        with open(f"carved_pe_{i}.bin", "wb") as out:
            out.write(data[offset:offset+2048])
        print(f"[+] PE carved at offset {offset}")

    # Step 2: Shellcode detection
    shell_regions = detect_shellcode_regions(data)
    for i, (start, _) in enumerate(shell_regions):
        with open(f"shellcode_{i}.bin", "wb") as out:
            out.write(data[start:start+512])
        print(f"[!] NOP sled/shellcode found at {start}")

    # Step 3: IOC extraction
    strings = extract_ascii_strings(data)
    with open("iocs_from_memory.txt", "w") as out:
        for s in strings:
```

```
            if ".onion" in s or "http" in s:
                out.write(s + "\n")

    # Step 4: Entropy scanning
    entropy_hits = scan_entropy_regions(data)
    for i, (offset, entropy) in enumerate(entropy_hits):
        with open(f"entropy_blob_{i}.bin", "wb") as out:
            out.write(data[offset:offset+512])
        print(f"[*] Saved high-entropy block at {offset}")

# Execute analysis
memory_carving_pipeline("memdump.raw")
```

Summary Table

Technique	Purpose
PE Header Search	Extract embedded executables
NOP Sled Detection	Find shellcode regions
ASCII String Extraction	Find plaintext IOCs
Regex Matching	Identify IPs/domains/passwords
Shannon Entropy Scan	Reveal packed/encrypted memory regions

5.8 - Automating Hybrid Analysis with Python

Objective

Hybrid Analysis is a malware sandbox that analyzes submitted files or URLs in a controlled environment and generates detailed reports with behavioral indicators, network activity, static attributes, and threat scores. This topic demonstrates how to automate:

- File submissions to Hybrid Analysis
- Retrieval of analysis reports
- Extraction of IOCs and key attributes
- Basic alerting and decision-making from the results

The automation helps security analysts process samples at scale or integrate threat analysis into pipelines.

---

1. Prerequisites

To follow this module, you'll need:

- A free Hybrid Analysis API key: https://www.hybrid-analysis.com/
- Python packages: `requests`, `time`, and optionally `pandas` or `rich` for reporting

Install requirements:

```
pip install requests
```

---

2. Setup and Authentication

We start by preparing the base URL and headers for interacting with the API.

```
import requests
import time
```

```python
# Replace with your actual Hybrid Analysis API key
API_KEY = 'your_api_key_here'

# Base headers for API calls
HEADERS = {
    'User-Agent': 'Falcon Sandbox',
    'api-key': API_KEY,
    'Content-Type': 'application/json'
}

BASE_URL = 'https://www.hybrid-analysis.com/api/v2'
```

---

3. Submitting a File for Analysis

We use the `/submit/file` endpoint to upload a suspicious file to the sandbox.

```python
def submit_file(filepath):
    print(f"[+] Submitting file: {filepath}")
    with open(filepath, 'rb') as f:
        files = {'file': f}
        data = {
            'environment_id': 300,  # 300 = Windows 10 64-bit
            'allow_dynamic': True,
            'priority': 1,
            'playbook': 'default'
        }
        response = requests.post(f"{BASE_URL}/submit/file", headers=HEADERS, data=data, files=files)

    if response.status_code == 200:
        submission_info = response.json()
```

```
        print(f"[+] Submission successful: {submission_info['job_id']}")
        return submission_info['job_id']
    else:
        print("[-] Submission failed:", response.text)
        return None
```

Note: You can choose different environments by changing `environment_id` (e.g., 120 = Windows 7 32-bit).

---

4. Polling the Analysis Status

Analysis may take time. We poll until the report is available.

```
def wait_for_analysis(job_id, timeout=300):
    print(f"[+] Waiting for analysis of job {job_id}")
    elapsed = 0
    while elapsed < timeout:
        response = requests.get(f"{BASE_URL}/search/hash/{job_id}", headers=HEADERS)
        if response.status_code == 200:
            results = response.json()
            if results:
                print("[+] Report is ready.")
                return results[0]
        time.sleep(15)
        elapsed += 15
    print("[-] Analysis timed out.")
    return None
```

---

5. Fetching the Final Report

Once available, we retrieve the behavioral report and network artifacts.

```
def get_report(report_sha256):
    print(f"[+] Fetching full report for hash: {report_sha256}")
    report_url = f"{BASE_URL}/report/{report_sha256}/summary"
    response = requests.get(report_url, headers=HEADERS)

    if response.status_code == 200:
        return response.json()
    else:
        print("[-] Failed to fetch report:", response.text)
        return None
```

---

6. Extracting and Displaying Key IOCs

We parse the report to extract high-confidence indicators such as:

- Contacted domains
- IP addresses
- Dropped files
- Processes
- Threat score

```
def parse_iocs(report):
    if not report:
        print("[-] No report to parse.")
        return

    print("\n[+] Parsed Threat Intelligence:")
```

```
    print(f"    Threat Score: {report['threat_score']}")
    print(f"    Verdict: {report.get('verdict', 'N/A')}")

    print("\n[+] Contacted Hosts:")
    for host in report.get("network", {}).get("hosts", []):
        print(f"    IP: {host.get('ip')}, Hostname: {host.get('hostname')}")

    print("\n[+] Domains:")
    for domain in report.get("network", {}).get("dns", []):
        print(f"    Domain: {domain.get('hostname')}")

    print("\n[+] Dropped Files:")
    for file in report.get("dropped", []):
        print(f"    {file.get('sha256')} - {file.get('type')} - {file.get('name')}")
```

---

7. Full Automation Pipeline

Here is a complete pipeline from submission to IOC extraction:

```
def analyze_file_pipeline(filepath):
    job_id = submit_file(filepath)
    if not job_id:
        return

    report_meta = wait_for_analysis(job_id)
    if not report_meta:
        return

    sha256 = report_meta.get("sha256")
    report = get_report(sha256)
    parse_iocs(report)
```

```
# Example run
analyze_file_pipeline("suspect_sample.exe")
```

---

8. Optional: Save Indicators to CSV

For later correlation or ingestion into SIEM tools.

```
import csv

def save_iocs_to_csv(report, filename="iocs.csv"):
    with open(filename, "w", newline='') as csvfile:
        writer = csv.writer(csvfile)
        writer.writerow(["Type", "Value"])

        for host in report.get("network", {}).get("hosts", []):
            writer.writerow(["IP", host.get("ip")])
        for domain in report.get("network", {}).get("dns", []):
            writer.writerow(["Domain", domain.get("hostname")])
        for file in report.get("dropped", []):
            writer.writerow(["Dropped SHA256", file.get("sha256")])
```

---

9. Summary

Component	Function
File Submission	Submits file to Hybrid Analysis sandbox

Component	Function
Polling	Waits for analysis job completion
Report Retrieval	Downloads summary of behavioral analysis
IOC Extraction	Extracts key threat indicators
CSV Export	Saves indicators for external use

---

10. Conclusion

Automating Hybrid Analysis with Python empowers analysts to:

- Integrate malware triage into pipelines
- Quickly assess large volumes of samples
- Extract actionable threat indicators
- Reduce manual investigation time

5.9 - Extracting Malware Configurations with Python

Objective:

Extracting configuration (C2 servers, campaign IDs, persistence methods, etc.) from malware samples is a critical step in threat intelligence and incident response. Many malware families embed their config data in cleartext or encoded formats. This topic focuses on how Python can automate the extraction of such embedded configurations from static malware samples using string search, regex, known offsets, deobfuscation, and pattern matching.

---

1. Introduction to Malware Configuration Extraction

Malware configuration (also known as malware "config") typically includes:

- C2 server domains or IP addresses
- Port numbers
- Campaign identifiers
- Mutexes
- File paths (e.g., persistence locations)
- User-Agent strings
- Sleep timers, keylogging flags, etc.

These values may be embedded as:

- Plaintext strings
- Base64-encoded blobs
- XOR-encrypted segments
- Custom obfuscated structures

---

2. Extracting Plaintext Strings

Many malware samples embed configs as plain strings. The `strings` tool or equivalent logic in Python can help extract them.

```
import re

def extract_ascii_strings(data, min_length=4):
    """
    Extracts ASCII strings from raw binary data.
    """
    pattern = rb"[ -~]{%d,}" % min_length  # Printable ASCII characters
    return re.findall(pattern, data)
```

```python
with open("sample_malware.bin", "rb") as f:
    binary = f.read()

strings_found = extract_ascii_strings(binary)
for s in strings_found:
    print(s.decode(errors="ignore"))
```

> Tip: This technique can help find plaintext domains, C2 addresses, and debug paths.

---

3. Identifying Base64-Encoded Configs

Malware sometimes uses Base64 to hide configurations. We can scan for likely Base64 strings and decode them.

```python
import base64

def find_base64_strings(strings_list):
    """
    Tries to decode Base64 strings from extracted data.
    """
    decoded_candidates = []
    for s in strings_list:
        try:
            decoded = base64.b64decode(s + b"===")  # Add padding
            if b"http" in decoded or b"." in decoded:
                decoded_candidates.append((s.decode(), decoded.decode(errors="ignore")))
        except:
            continue
    return decoded_candidates

decoded_configs = find_base64_strings(strings_found)
```

```python
for raw, decoded in decoded_configs:
    print(f"Base64: {raw} -> {decoded}")
```

---

4. XOR Obfuscated Config Extraction

A common evasion technique is XOR-ing the config with a static key.

```python
def xor_decrypt(data, key):
    """
    Decrypts XOR-obfuscated data with a repeating key.
    """
    return bytes([b ^ key[i % len(key)] for i, b in enumerate(data)])

# Example usage
# Let's assume we extracted a blob or a config offset (just for simulation here)
xor_key = b"\x12\x34"
encrypted_blob = b"\x77\x50\x45\x63"  # XORed bytes (example)

decrypted = xor_decrypt(encrypted_blob, xor_key)
print("Decrypted:", decrypted)
```

> You can also brute-force short keys and look for readable content like URLs or IPs in the decrypted result.

---

5. Regex-Based Config Extraction from PE or Script Files

Often, the malware config is hardcoded in Python scripts (e.g., in malware like AgentTesla droppers, Ransomware kits).

```python
sample_script = """
url = "http://malicious.site/connect"
port = 443
keylog = True
"""

# Pattern for extracting config values
pattern = r'url\s*=\s*["\'](.*?)["\']'

match = re.search(pattern, sample_script)
if match:
    print("C2 URL found:", match.group(1))
```

You can extend this technique to extract multiple fields:

```python
patterns = {
    "url": r'url\s*=\s*["\'](.*?)["\']',
    "port": r'port\s*=\s*(\d+)',
    "keylog": r'keylog\s*=\s*(True|False)'
}

for key, pat in patterns.items():
    match = re.search(pat, sample_script)
    if match:
        print(f"{key} => {match.group(1)}")
```

---

6. Known Offset / Static Signature Extraction

If the malware has a known config structure (e.g., starts at byte 0x200 with fixed format), use slicing.

```
config_offset = 0x200
config_length = 100

config_data = binary[config_offset : config_offset + config_length]
print("Possible config area:\n", config_data)
```

You can then pass this to a parser (string decoder, struct unpacker, etc.) based on your malware family.

---

7. Example: Extracting Config from Emotet or LokiBot

For families like Emotet or Lokibot, config extraction often includes:

- Domain list (null-separated)
- XOR key
- Campaign ID
- Mutex string

```
def extract_domains_null_terminated(blob):
    return [d for d in blob.split(b'\x00') if b'.' in d]

# Simulated XORed blob
raw_config = xor_decrypt(binary[0x300:0x400], b'\x56')

# Try to extract domains
domains = extract_domains_null_terminated(raw_config)
for d in domains:
    print("Domain:", d.decode(errors="ignore"))
```

---

8. Reporting and Structuring Extracted Config

Once you extract indicators, you can structure and export them:

```
import json

malware_config = {
    "c2_servers": domains,
    "xor_key": "56",
    "extracted_from": "offset_0x300"
}

with open("extracted_config.json", "w") as f:
    json.dump(malware_config, f, indent=2)
```

---

9. Summary Table

Technique	When to Use
ASCII strings extraction	Plain embedded values (URLs, IPs)
Base64 detection	Light obfuscation of strings
XOR decryption	Lightweight encryption schemes
Regex parsing	Script-based or visible malware code
Offset-based slicing	Static binaries with known layout

5.10 - Building a YARA Scanner in Python

Objective:

Enable cybersecurity professionals to integrate YARA rule scanning directly within Python workflows for detection, classification, and threat hunting of malicious artifacts. This includes writing custom rules, loading them in Python, scanning memory or disk files, and handling matches programmatically.

---

1. What is YARA?

YARA is a pattern-matching engine used to identify malware families and suspicious files. It works by applying user-defined rules against binary data or memory.

Each rule contains:

- **Strings to search for**
- **Conditions under which to match**
- **Optional metadata (author, description, etc.)**

YARA is ideal for:

- Detecting known malware variants
- Classifying unknown samples
- Tagging binaries or files for triage

---

2. Installing YARA and Python Bindings

You need to install both the core YARA engine and its Python bindings:

```
pip install yara-python
```

Or, if you need to build from source:

```
git clone https://github.com/VirusTotal/yara
cd yara
./bootstrap.sh && ./configure && make && sudo make install
pip install yara-python
```

---

3. Writing a Basic YARA Rule

Let's write a simple YARA rule that detects a known signature (like a string or hex pattern):

```
rule SuspiciousExecutable
{
    meta:
        description = "Detects EXE files with suspicious imports"
        author = "Analyst"
        threat_level = 3

    strings:
        $a = "VirtualAlloc"
        $b = "LoadLibrary"
        $c = { 6A 00 68 ?? ?? ?? ?? E8 ?? ?? ?? ?? }

    condition:
        2 of ($a,$b,$c)
}
```

Save this rule in a file like `rules.yar`.

---

4. Loading and Compiling YARA Rules in Python

```python
import yara

# Compile the rule file into memory
rules = yara.compile(filepath='rules.yar')

# Load and scan a file (e.g., a malware sample)
matches = rules.match('suspect.exe')

# Print out the matched rule names and metadata
for match in matches:
    print(f"[+] Rule matched: {match.rule}")
    for name, value in match.meta.items():
        print(f"    {name}: {value}")
```

> 🔍 **Note**: You can also compile from string using `yara.compile(source=...)`.

---

5. Handling Rule Matches Programmatically

YARA results can be parsed and used to trigger alerts, store indicators, or classify samples.

```python
def analyze_yara_output(matches):
    result = {}
    for match in matches:
        result[match.rule] = {
            "meta": match.meta,
            "strings_matched": [s[1] for s in match.strings],
        }
    return result
```

```
summary = analyze_yara_output(matches)
print(summary)
```

This approach allows you to log or forward structured information to SIEMs or dashboards.

---

6. Scanning In-Memory Data

You can scan memory buffers (like extracted config sections or decrypted payloads):

```
buffer_data = b"This contains VirtualAlloc and LoadLibrary and some shellcode..."

matches = rules.match(data=buffer_data)
for match in matches:
    print(f"Match in buffer: {match.rule}")
```

Useful for sandboxing, unpacked payloads, or dynamic analysis outputs.

---

7. Compiling Multiple Rule Files and Rule Sets

```
rules = yara.compile(filepaths={
    'shellcode_rules': 'shellcode.yar',
    'persistence_rules': 'persistence.yar',
    'obfuscation_rules': 'obfuscation.yar'
})
```

This lets you maintain modular, targeted rulebases by category.

8. Creating Dynamic Rules in Python

If you're building signatures dynamically:

```
rule_template = '''
rule DynamicDomainRule {{
    meta:
        source = "ThreatFeed"
    strings:
        $domain = "{domain}"
    condition:
        $domain
}}
'''

# Dynamically build rule for a domain
new_rule = rule_template.format(domain="malicious-site.com")
rules = yara.compile(source=new_rule)

matches = rules.match(data=b"Connecting to malicious-site.com ...")
for m in matches:
    print(f"[Dynamic] Matched rule: {m.rule}")
```

Great for IOC feeds or real-time alert enrichment.

---

9. Error Handling with Invalid Rules

```
try:
    rules = yara.compile(filepath="broken_rules.yar")
```

```
except yara.SyntaxError as e:
    print("YARA syntax error:", e)
```

Always validate rules before loading, especially if sourced from external systems or feeds.

---

10. Integration Use Cases

Use Case	How YARA Helps
Triage of suspicious files	Classify and tag malware families
Threat hunting	Scan disk/memory for signatures of known malware
IOC correlation	Match domains, mutexes, hashes in forensics data
Sandbox post-processing	Scan unpacked artifacts or memory dumps
Alert enrichment in SIEMs	Add matched YARA rules to detection context

---

11. Outputting Results to JSON/CSV

```
import json

results = analyze_yara_output(matches)
with open("yara_results.json", "w") as f:
    json.dump(results, f, indent=2)
```

Or CSV:

```python
import csv

with open("yara_matches.csv", "w", newline='') as f:
    writer = csv.writer(f)
    writer.writerow(["Rule", "Matched Strings"])
    for match in matches:
        strings = ','.join([s[1] for s in match.strings])
        writer.writerow([match.rule, strings])
```

---

12. Summary: YARA via Python

Pros:

- Flexible rule execution across memory, files, buffers
- Integration with automation workflows
- Compatible with IOC enrichment, triage, and hunting tools

Best Practices:

- Regularly validate rule syntax
- Avoid overly generic rules (false positives)
- Tune YARA conditions for performance

5.11 - Integrating Python with Cuckoo Sandbox for Dynamic Analysis

Objective:

Teach how to automate malware analysis workflows by interacting with Cuckoo Sandbox using Python. This includes submitting files, retrieving analysis results, parsing behavior logs, and extracting IOCs.

1. Introduction to Cuckoo Sandbox

Cuckoo Sandbox is an open-source automated malware analysis system. It can run suspicious files inside virtual environments (like VirtualBox or KVM) and produce detailed reports of their behavior:

- API calls
- Network traffic
- File system changes
- Process creation
- Registry modifications

Cuckoo outputs results as JSON, allowing programmatic access via Python for further correlation, enrichment, or detection pipelines.

---

2. Setting Up the Environment

Before using Python to interact with Cuckoo, make sure you:

- Have Cuckoo Sandbox installed and running (`cuckoo` and `cuckoo web`).
- Have configured the API module (`cuckoo api` on Cuckoo 2.x).
- Can access the API (usually at `http://localhost:8090`).

You can test connectivity with:

```
curl http://localhost:8090/tasks/list
```

---

3. Submitting a File for Analysis via API

```python
import requests

CUCKOO_API = "http://localhost:8090/tasks/create/file"

# Prepare the file for submission
with open("sample.exe", "rb") as f:
    files = {"file": ("sample.exe", f)}

# Submit the file to Cuckoo
response = requests.post(CUCKOO_API, files=files)

# Parse and print the task ID
if response.ok:
    task_id = response.json()["task_id"]
    print(f"Submitted successfully. Task ID: {task_id}")
else:
    print(f"Error: {response.status_code} - {response.text}")
```

Explanation:

- The file is submitted as multipart/form-data.
- The API returns a JSON response containing the `task_id`, which is used to track the analysis.

---

4. Checking the Status of the Analysis

```python
import time

def check_status(task_id):
```

```
    url = f"http://localhost:8090/tasks/view/{task_id}"
    while True:
        resp = requests.get(url)
        status = resp.json()["task"]["status"]
        print(f"Current Status: {status}")
        if status == "reported":
            break
        time.sleep(5)

task_id = 1  # Replace with your actual task ID
check_status(task_id)
```

Explanation:

- This function polls Cuckoo every 5 seconds until the analysis is complete.
- Status options include: `pending`, `running`, `completed`, and `reported`.

---

5. Retrieving the Full Analysis Report (JSON)

```
def get_report(task_id):
    url = f"http://localhost:8090/tasks/report/{task_id}"
    response = requests.get(url)
    if response.ok:
        return response.json()
    else:
        raise Exception(f"Failed to get report: {response.status_code}")

report = get_report(task_id)
```

Explanation:

- The full report includes behavioral logs, dropped files, signatures, network activity, etc.

---

6. Parsing Behavioral IOCs from the Report

```python
def extract_iocs(report):
    iocs = {
        "domains": [],
        "ips": [],
        "mutexes": [],
        "executed_commands": []
    }

    # Network section
    for domain in report.get("network", {}).get("domains", []):
        iocs["domains"].append(domain["domain"])

    for host in report.get("network", {}).get("hosts", []):
        iocs["ips"].append(host["ip"])

    # Behavioral section
    for process in report.get("behavior", {}).get("processes", []):
        for call in process.get("calls", []):
            if call["api"] == "CreateMutexW":
                arguments = call.get("arguments", [])
                for arg in arguments:
                    if arg["name"] == "MutexName":
                        iocs["mutexes"].append(arg["value"])
            elif call["api"] == "WinExec":
                for arg in call.get("arguments", []):
                    if arg["name"] == "CommandLine":
                        iocs["executed_commands"].append(arg["value"])
```

```
    return iocs

parsed_iocs = extract_iocs(report)
print("Extracted IOCs:")
for key, values in parsed_iocs.items():
    print(f"{key.upper()}: {values}")
```

Explanation:

- Extracts indicators from both network and behavioral sections.
- Looks for API call arguments like `CreateMutexW` and `WinExec`.

---

7. Exporting Results to JSON/CSV for External Use

```
import json

with open("cuckoo_iocs.json", "w") as f:
    json.dump(parsed_iocs, f, indent=2)
```

Or to CSV:

```
import csv

with open("cuckoo_iocs.csv", "w", newline='') as f:
    writer = csv.writer(f)
    writer.writerow(["Type", "Value"])
    for ioc_type, items in parsed_iocs.items():
```

```
        for item in items:
            writer.writerow([ioc_type, item])
```

---

8. Automating the Full Workflow

You can wrap the entire workflow in a function:

```
def analyze_sample(file_path):
    with open(file_path, "rb") as f:
        response = requests.post(CUCKOO_API, files={"file": (file_path, f)})

    task_id = response.json()["task_id"]
    print(f"Submitted Task ID: {task_id}")

    check_status(task_id)
    report = get_report(task_id)
    return extract_iocs(report)

iocs = analyze_sample("sample.exe")
print(json.dumps(iocs, indent=2))
```

---

9. Use Cases of Python + Cuckoo Integration

Use Case	Description
Automated triage pipeline	Auto-submit quarantined files and tag them with IOCs
IOC extraction & correlation	Integrate with MISP/Elastic/Splunk for threat hunting

Use Case	Description
Dynamic YARA + Cuckoo	Match behaviors with custom YARA + API correlation
Alert enrichment in SOC	Submit hashes or payloads received from alerts

---

10. Summary

Integrating Python with Cuckoo Sandbox allows:

- Seamless, scalable malware analysis workflows
- Automated detection of behavioral indicators
- Programmatic access to sandboxed execution logs
- Rapid export and sharing of results across tools

5.12 - Using Python to Extract Embedded Shellcode or Payloads from Binaries

Objective:

This topic teaches how to analyze binary files (such as malware samples, dropper EXEs, packed files) to extract embedded shellcode or payloads using Python. The focus is on static binary inspection, pattern searching, entropy analysis, and common shellcode detection heuristics.

---

1. Introduction to Embedded Payload Extraction

Malware often hides or embeds payloads inside executables, DLLs, scripts, or even document files. These payloads could be:

- Raw shellcode (binary)
- Base64/XOR encoded payloads

- Encrypted payloads (e.g., AES)
- Packed inside resource or data sections

Python allows us to programmatically analyze binaries to:

- Detect suspicious byte patterns
- Identify NOP sleds, syscall markers, shellcode stubs
- Extract payloads into separate files for further analysis or execution in a safe environment

---

2. Reading Raw Binary Data in Python

```python
def read_binary(file_path):
    with open(file_path, "rb") as f:
        data = f.read()
    return data

# Load the binary into memory
binary_data = read_binary("suspicious_sample.exe")
print(f"Binary loaded: {len(binary_data)} bytes")
```

Explanation:

- Binary is read as raw bytes (`rb` mode).
- Data is loaded into memory for in-memory inspection (important for shellcode detection).

---

3. Searching for Common Shellcode Patterns (Heuristics)

A common technique is searching for NOP sleds (`\x90`) or syscall instructions (`\xCD\x80` , `\x0F\x05`).

```
def find_shellcode_patterns(data):
    matches = []

    # Simple heuristic: NOP sled + suspicious size
    pattern = b"\x90" * 10  # 10 or more NOPs in a row
    index = 0

    while True:
        index = data.find(pattern, index)
        if index == -1:
            break
        # Extract 300 bytes from that point (potential shellcode)
        snippet = data[index:index + 300]
        matches.append((index, snippet))
        index += 10  # Move forward to find next match

    return matches

results = find_shellcode_patterns(binary_data)
print(f"Found {len(results)} potential shellcode segments.")
```

Explanation:

- Looks for a sequence of NOPs (often used as sleds before real shellcode).
- Captures a 300-byte snippet starting from the match (can be adjusted).
- This is a *heuristic*, not a guarantee — requires manual or automated disassembly later.

---

4. Saving Extracted Payloads to Disk

```python
import os

def save_payloads(matches, output_dir="extracted_payloads"):
    os.makedirs(output_dir, exist_ok=True)
    for i, (offset, payload) in enumerate(matches):
        filename = f"{output_dir}/payload_{i}_at_{offset}.bin"
        with open(filename, "wb") as f:
            f.write(payload)
        print(f"[+] Saved {filename} ({len(payload)} bytes)")

save_payloads(results)
```

Explanation:

- Each shellcode segment found is written to a `.bin` file.
- You can later load this into tools like `radare2`, `IDA`, or a custom emulator for analysis.

---

5. Advanced Technique: Entropy Analysis

Shellcode is typically high-entropy compared to regular code/data.

```python
import math

def calculate_entropy(data):
    if not data:
        return 0
    byte_counts = [0] * 256
    for b in data:
        byte_counts[b] += 1
    entropy = 0
```

```
    for count in byte_counts:
        if count == 0:
            continue
        p = count / len(data)
        entropy -= p * math.log2(p)
    return entropy

# Sliding window entropy analysis
window_size = 512
threshold = 7.5  # High-entropy threshold
high_entropy_regions = []

for i in range(0, len(binary_data) - window_size, 100):
    window = binary_data[i:i + window_size]
    entropy = calculate_entropy(window)
    if entropy > threshold:
        high_entropy_regions.append((i, entropy))

print(f"High-entropy regions found: {len(high_entropy_regions)}")
```

Explanation:

- Slides a window through the binary.
- Computes Shannon entropy (values close to 8.0 = high randomness).
- Useful for identifying encrypted or packed sections.

---

6. Extracting Payloads Based on Entropy

You can combine entropy with other heuristics:

```python
def extract_high_entropy_regions(data, regions, window_size=512):
    payloads = []
    for offset, _ in regions:
        snippet = data[offset:offset + window_size]
        payloads.append((offset, snippet))
    return payloads

entropy_payloads = extract_high_entropy_regions(binary_data, high_entropy_regions)
save_payloads(entropy_payloads, output_dir="entropy_payloads")
```

---

7. Optional: Signature Matching (YARA Style)

You can also use byte pattern matching (poor man's YARA):

```python
import re

# Example: detect msfvenom shellcode stubs (Windows exec)
pattern = b"\xfc\xe8[\x00-\xff]{4}\x60\x89\xe5"

matches = list(re.finditer(pattern, binary_data))
print(f"Signature matches: {len(matches)}")
```

---

8. Summary of Best Practices

Technique	Use Case
NOP sled detection	Find simple shellcode or stagers

Technique	Use Case
Entropy analysis	Detect packed/encrypted sections
Sliding window search	Cover the full binary file
Regex signature search	Match known shellcode patterns
Output segmentation	Isolate each match for tool-based review

---

9. Tools to Combine With

- **Capstone Engine**: Disassemble extracted shellcode
- **Unicorn Engine**: Emulate shellcode execution
- **radare2 / Cutter**: Visual inspection of extracted `.bin`
- **VirusTotal / HybridAnalysis**: Submit extracted payloads
- **YARA**: Write rules to detect patterns at scale

5.13 - Analyzing Malware Network Behavior with Python (PCAP Parsing and Indicators)

Objective:

To equip analysts with the ability to analyze malware network activity by parsing PCAP files, extracting indicators of compromise (IOCs) such as domains, IPs, and URLs, and identifying suspicious communication patterns using Python.

---

1. Introduction

When malware executes, it often generates network traffic that includes DNS queries, HTTP requests, C2 beacons, file downloads, and exfiltration of data. These behaviors are captured in PCAP (Packet Capture) files using tools like Wireshark or tcpdump.

Python provides a powerful ecosystem for processing and analyzing PCAP files:

- Identifying domains and IPs contacted
- Extracting HTTP URLs and headers
- Correlating timestamps and packet size
- Flagging beaconing or encoded traffic

We'll use the `scapy` and `pyshark` libraries for this purpose.

---

2. Preparing the Environment

Install required libraries:

```
pip install scapy pyshark
```

---

3. Parsing PCAP Files with Scapy

```
from scapy.all import rdpcap

# Load packets from PCAP file
packets = rdpcap("malware_traffic.pcap")

print(f"Total packets loaded: {len(packets)}")
```

Explanation:

- `rdpcap` reads all packets from a `.pcap` file.

- We now have a list of raw packets for inspection.

---

4. Extracting IP Addresses and Communication Flows

```
from collections import defaultdict

ip_communications = defaultdict(int)

for pkt in packets:
    if pkt.haslayer("IP"):
        src = pkt["IP"].src
        dst = pkt["IP"].dst
        ip_communications[(src, dst)] += 1  # Track how many times src talked to dst

# Print top communications
for (src, dst), count in sorted(ip_communications.items(), key=lambda x: -x[1])[:10]:
    print(f"{src} -> {dst} : {count} packets")
```

Explanation:

- Tracks communication flows between source and destination IPs.
- Useful to find the most active connections or lateral movement.

---

5. Extracting DNS Queries (Potential C2 Domains)

```
from scapy.layers.dns import DNSQR
```

```
dns_queries = set()

for pkt in packets:
    if pkt.haslayer("DNS") and pkt.haslayer("UDP"):
        if pkt["DNS"].qd is not None:
            query = pkt["DNS"].qd.qname.decode().strip(".")
            dns_queries.add(query)

print("[*] Unique DNS Queries:")
for domain in sorted(dns_queries):
    print(" -", domain)
```

Explanation:

- Extracts queried domains (C2 beaconing, DGA detection).
- Use for IOC enrichment and blacklist comparison.

---

6. Extracting HTTP Requests with PyShark

```
import pyshark

cap = pyshark.FileCapture("malware_traffic.pcap", display_filter="http")

for pkt in cap:
    try:
        host = pkt.http.host
        uri = pkt.http.request_uri
        method = pkt.http.request_method
        full_url = f"{method} http://{host}{uri}"
        print(full_url)
```

```
    except AttributeError:
        continue
```

Explanation:

- Parses full HTTP requests (methods, URIs, hosts).
- Ideal for identifying malware C2 URLs or malicious download links.

---

7. Detecting Base64 or Hex in URLs

```
import re

def is_base64_or_hex(s):
    base64_pattern = r"([A-Za-z0-9+/]{20,}={0,2})"
    hex_pattern = r"([A-Fa-f0-9]{16,})"
    return re.search(base64_pattern, s) or re.search(hex_pattern, s)

suspicious_urls = []

for pkt in cap:
    try:
        uri = pkt.http.request_uri
        if is_base64_or_hex(uri):
            suspicious_urls.append(uri)
    except:
        continue

print("[*] Suspicious URLs with base64 or hex-like patterns:")
for url in suspicious_urls:
    print(" -", url)
```

Explanation:

- Finds URLs with embedded encoded data.
- Could indicate malware beacons or exfiltration attempts.

---

8. Extracting Timestamps and Identifying Beaconing

```
from datetime import datetime

c2_timeline = []

for pkt in cap:
    try:
        timestamp = pkt.sniff_time
        ip = pkt.ip.dst
        host = pkt.http.host
        c2_timeline.append((timestamp, ip, host))
    except:
        continue

# Print timeline of C2 contact
for ts, ip, host in sorted(c2_timeline):
    print(f"[{ts}] → {ip} ({host})")
```

Explanation:

- Collects timestamps of outbound connections.
- Useful for identifying regular time-based patterns (beaconing).

---

9. Summary Table of IOCs

```
import json

ioc_report = {
    "domains": list(sorted(dns_queries)),
    "suspicious_urls": suspicious_urls,
    "ip_communications": [{"src": src, "dst": dst, "count": count}
                          for (src, dst), count in ip_communications.items()]
}

with open("ioc_report.json", "w") as f:
    json.dump(ioc_report, f, indent=2)

print("IOC report saved to ioc_report.json")
```

Explanation:

- Consolidates all extracted indicators into structured JSON.
- Ready for integration into dashboards or SIEMs.

---

10. Optional: Detecting Unusual Ports

```
from scapy.layers.inet import TCP, UDP

ports_used = defaultdict(int)

for pkt in packets:
    if pkt.haslayer(TCP):
        ports_used[pkt[TCP].dport] += 1
```

```
    elif pkt.haslayer(UDP):
        ports_used[pkt[UDP].dport] += 1

# Print uncommon ports (e.g., >1024)
for port, count in sorted(ports_used.items()):
    if port > 1024:
        print(f"Port {port} → {count} packets")
```

5.14 - Detecting Anti-Analysis and Evasion Techniques in Malware using Python

Objective:

Understand and detect common anti-analysis and evasion techniques used by malware to avoid sandboxing, reverse engineering, and automated detection — using Python scripts and static/dynamic heuristics.

---

1. Introduction

Modern malware frequently includes anti-analysis features to detect virtual environments, debugging tools, sandbox emulators, and user inactivity. Detecting these techniques helps analysts identify malware attempting to hide its true behavior.

This module will focus on:

- Static detection of evasion functions and strings
- Identifying suspicious API calls and system functions
- Recognizing timing-based evasion (sleep, loops)
- Behavioral patterns for sandbox detection
- Registry and process inspection for anti-debugging

---

2. Static Detection of Anti-VM Strings

```python
def detect_vm_indicators(file_path):
    """
    Scans file contents for common anti-VM strings.
    """
    vm_indicators = [
        "VBox", "VMware", "vboxservice", "vboxtray",
        "qemu", "VirtualBox", "virtual_machine",
        "vmtools", "vmguest", "Xen"
    ]

    with open(file_path, "rb") as f:
        content = f.read().decode(errors="ignore")

    matches = [s for s in vm_indicators if s.lower() in content.lower()]

    if matches:
        print("Detected possible anti-VM techniques:")
        for m in matches:
            print(" -", m)
    else:
        print("No anti-VM indicators found.")
```

Explanation:

- Malware often searches for virtualization-related strings.
- We search for those hardcoded in the binary.

---

3. Detecting Timing-Based Evasion (Sleep, GetTickCount)

```python
def detect_timing_api_calls(file_path):
    """
    Searches for API calls known for delaying execution to bypass sandboxes.
    """
    timing_functions = [
        "Sleep", "GetTickCount", "QueryPerformanceCounter",
        "NtDelayExecution", "RDTSC", "timeGetTime"
    ]

    with open(file_path, "rb") as f:
        content = f.read().decode(errors="ignore")

    found = [api for api in timing_functions if api.lower() in content.lower()]

    if found:
        print("Timing-based evasion API calls found:")
        for api in found:
            print(" -", api)
```

Explanation:

- Sandboxes often run malware for limited time (e.g., 10 seconds).
- Malware delays malicious execution with time-based functions.

---

4. Detecting User Interaction Checks (Anti-Sandbox)

```python
def detect_user_interaction_checks(file_path):
    """
    Scans for functions used to detect human interaction.
    """
```

```
    interaction_apis = [
        "GetCursorPos", "GetAsyncKeyState", "GetForegroundWindow",
        "GetLastInputInfo", "ShowCursor", "mouse_event", "keybd_event"
    ]

    with open(file_path, "rb") as f:
        content = f.read().decode(errors="ignore")

    detected = [fn for fn in interaction_apis if fn.lower() in content.lower()]

    if detected:
        print("Possible user interaction checks detected:")
        for fn in detected:
            print(" -", fn)
```

Explanation:

- These functions detect whether a real user is present.
- Sandboxes may not emulate mouse/keyboard properly.

---

5. Detecting Debugger Detection Techniques

```
def detect_debugger_checks(file_path):
    """
    Identifies common anti-debugging API calls and tricks.
    """
    debugger_apis = [
        "IsDebuggerPresent", "CheckRemoteDebuggerPresent",
        "NtQueryInformationProcess", "OutputDebugString",
        "FindWindow", "CloseHandle"
```

```
    ]

    with open(file_path, "rb") as f:
        content = f.read().decode(errors="ignore")

    flags = [api for api in debugger_apis if api.lower() in content.lower()]

    if flags:
        print("Debugger detection APIs found:")
        for api in flags:
            print(" -", api)
```

Explanation:

- Malware may detect tools like OllyDbg, x64dbg, IDA.
- These API calls are classic anti-debug tricks.

---

6. Entropy-Based Obfuscation Detection

```
import math

def calculate_entropy(data):
    """
    Calculate Shannon entropy to estimate randomness/obfuscation.
    """
    if not data:
        return 0
    entropy = 0
    for x in range(256):
        p_x = data.count(bytes([x])) / len(data)
```

```
        if p_x > 0:
            entropy -= p_x * math.log2(p_x)
    return entropy

def scan_entropy(file_path):
    """
    Detects high-entropy sections in binary files, which may indicate packing or encryption.
    """
    with open(file_path, "rb") as f:
        content = f.read()

    window = 1024
    threshold = 7.5  # Normal code/text entropy is ~5; >7 is suspicious

    for i in range(0, len(content) - window, window):
        chunk = content[i:i+window]
        ent = calculate_entropy(chunk)
        if ent > threshold:
            print(f"High entropy detected at offset {i:#x}, entropy = {ent:.2f}")
```

Explanation:

- High entropy suggests encrypted or packed payloads.
- Many malware samples use packing (UPX, custom crypters).

---

7. Process Hollowing or Injection Detection Heuristics

While this requires live monitoring (e.g., via `psutil` or ETW), a simple detection of code injection strings can be done statically.

```
def detect_injection_indicators(file_path):
    """
```

```
    Look for Windows APIs used in process injection techniques.
    """
    injection_apis = [
        "OpenProcess", "VirtualAllocEx", "WriteProcessMemory",
        "CreateRemoteThread", "NtCreateThreadEx", "SetThreadContext"
    ]

    with open(file_path, "rb") as f:
        content = f.read().decode(errors="ignore")

    hits = [api for api in injection_apis if api.lower() in content.lower()]

    if hits:
        print("Potential process injection APIs found:")
        for api in hits:
            print(" -", api)
```

---

8. Summary and Report

```
def run_evasion_analysis(file_path):
    print("== Anti-Analysis & Evasion Check ==")
    detect_vm_indicators(file_path)
    detect_timing_api_calls(file_path)
    detect_user_interaction_checks(file_path)
    detect_debugger_checks(file_path)
    scan_entropy(file_path)
    detect_injection_indicators(file_path)

# Example usage:
# run_evasion_analysis("suspect.exe")
```

9. Conclusion

By automating the detection of evasion and anti-analysis techniques, security teams can:

- Identify sophisticated malware early in the analysis
- Avoid sandbox evasion pitfalls
- Prioritize samples for dynamic analysis
- Feed rules into YARA or AV detection

5.15 - Memory Dump Analysis and IOC Extraction with Python

Objective:

Learn how to analyze memory dumps using Python for the purpose of extracting Indicators of Compromise (IOCs), such as domains, IPs, URLs, file paths, and suspicious strings. The focus is on automated triage and analysis for malware memory forensics.

---

1. Introduction to Memory Dump Analysis

Memory dumps are snapshots of the physical RAM of a system, often captured during incident response or malware execution. They can reveal:

- In-memory malware payloads
- Decrypted strings (not found in static binaries)
- C2 URLs or IPs used by malware
- Shellcode and injected code

Python can be used to parse, search, and extract meaningful artifacts from these dumps.

---

2. Prerequisites and Setup

We'll work with:

- Raw `.dmp` or `.bin` memory dumps (e.g., from tools like DumpIt, WinPMEM)
- Pure Python libraries (`re` , `os` , `struct`) or integration with tools like **Volatility** for deeper inspection

```
# Example memory dump collection tool (outside Python):
# winpmem -o memory.raw
```

---

3. IOC Patterns for Extraction

Define a basic list of regex patterns to extract IOCs commonly found in memory:

```
import re

# Define regex patterns for IOC types
ioc_patterns = {
    "ipv4": re.compile(r"\b(?:[0-9]{1,3}\.){3}[0-9]{1,3}\b"),
    "domain": re.compile(r"\b(?:[a-z0-9](?:[a-z0-9\-]{0,61}[a-z0-9])?\.)+(?:[a-z]{2,})\b", re.IGNORECASE),
    "url": re.compile(r"https?://[^\s\"']+", re.IGNORECASE),
    "email": re.compile(r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b"),
    "filepath": re.compile(rb"[A-Z]:\\[^:*?\"<>|\r\n]+")
}
```

Explanation:

- IPs, domains, URLs, filepaths, and emails are common artifacts.
- Regex is the fastest way to locate them in raw text buffers from memory.

4. Read and Scan a Memory Dump

```python
def scan_memory_dump(file_path):
    """
    Reads a memory dump file and extracts IOCs using regex patterns.
    """
    results = {key: set() for key in ioc_patterns}

    with open(file_path, "rb") as f:
        data = f.read()

        # Convert binary to string safely for regex matching
        ascii_data = data.decode(errors="ignore")

        # Scan ASCII string content
        for key, pattern in ioc_patterns.items():
            if key != "filepath":
                matches = pattern.findall(ascii_data)
            else:
                # Filepath regex works better in raw binary mode
                matches = pattern.findall(data)
                matches = [m.decode(errors="ignore") for m in matches]

            results[key].update(matches)

    return results
```

5. Display the Results

```python
def display_ioc_results(results):
    print("=== IOC Extraction Results ===")
    for ioc_type, values in results.items():
        print(f"\n{ioc_type.upper()} ({len(values)} found):")
        for val in sorted(values):
            print(" -", val)
```

---

6. Full Execution Example

```python
if __name__ == "__main__":
    file_path = "memory_dump.raw"  # Replace with your actual dump file
    results = scan_memory_dump(file_path)
    display_ioc_results(results)
```

---

7. Enhancing with Context: Nearby Strings

You can extract additional context surrounding the IOCs:

```python
def extract_context(data, match, window=40):
    """
    Extracts context around an IOC match (e.g., 40 bytes before and after).
    """
    index = data.find(match.encode())
    if index != -1:
        start = max(0, index - window)
        end = index + len(match) + window
```

```
        return data[start:end].decode(errors="ignore", errors="ignore")
    return ""
```

This helps analysts determine whether a URL or domain is actively used or referenced by malware.

---

8. Optional: Integration with Volatility Framework

For more advanced memory parsing (processes, network sockets, DLLs), use Volatility:

```
# Example: List processes with Volatility
volatility3 -f memory.raw windows.pslist
```

To automate Volatility in Python:

```
import subprocess

def run_volatility(plugin, dump_file):
    """
    Wrapper to execute a Volatility plugin and return its output.
    """
    cmd = ["volatility3", "-f", dump_file, plugin]
    result = subprocess.run(cmd, capture_output=True, text=True)
    return result.stdout
```

---

9. Exporting IOCs to JSON

```python
import json

def export_to_json(results, output_file="iocs_extracted.json"):
    with open(output_file, "w") as f:
        json.dump(results, f, indent=2)
    print(f"\n[+] IOCs exported to {output_file}")
```

---

10. Conclusion

Memory dump analysis enables deep insight into malware behavior and runtime IOCs not visible in binaries.

Python can:

- Quickly extract key indicators from raw dumps
- Identify encrypted payloads in memory
- Integrate with Volatility or YARA
- Enable scalable triage of multiple memory samples

5.16 - YARA Rule Matching in Memory and Files with Python

Objective

Understand how to use Python to interface with YARA rules for identifying patterns associated with malware and IOCs in memory dumps and files. We'll cover rule creation, compilation, scanning files and memory, handling matches, and best practices.

---

1. Introduction to YARA

YARA is a pattern matching engine primarily used in malware research and threat detection. It allows analysts to define *rules* to detect file signatures, strings, byte patterns, and behaviors.

Python supports YARA via the `yara-python` library, which binds directly to YARA's C library.

---

2. Installing YARA and Python Bindings

Install YARA and the Python bindings using pip:

```
pip install yara-python
```

Ensure that the system YARA library is also installed (on some Linux systems you might need `libyara-dev`).

---

3. Basic YARA Rule Structure

A YARA rule has:

- A **name**
- **Strings** to match (text, hex, regex)
- A **condition** to trigger a match

Example YARA rule to detect a malicious mutex or function:

```
rule SuspiciousFunction
{
    strings:
        $a = "CreateRemoteThread"
        $b = "VirtualAllocEx"
```

```
        $c = /malicious_\w{4}/

    condition:
        2 of them
}
```

---

4. Compiling YARA Rules in Python

```python
import yara

# Define YARA rules as string
rules_text = """
rule MalwarePattern
{
    strings:
        $a = "cmd.exe"
        $b = "powershell.exe"
        $hex = { 6A 40 68 00 30 00 00 }  // Common PE shellcode pattern

    condition:
        any of them
}
"""

# Compile the rule into an executable form
rules = yara.compile(source=rules_text)
```

🔍 Commentary:

- The `source=` argument compiles a YARA rule directly from a Python string.

- You can also use `filename="rules.yar"` to load from a `.yar` file.

---

5. Scanning Files with YARA

```
def scan_file(filepath, compiled_rules):
    """
    Scan a file using compiled YARA rules.
    """
    matches = compiled_rules.match(filepath)
    return matches
```

🔍 Commentary:

- `match()` scans the file and returns a list of matches (if any).
- Each match includes rule name, strings matched, and offsets.

---

6. Example: Full Scan Workflow

```
if __name__ == "__main__":
    target_file = "suspicious_file.bin"  # Replace with your sample

    try:
        results = scan_file(target_file, rules)

        if results:
            print("[+] Matches found:")
            for match in results:
                print("Rule:", match.rule)
```

```
                for s in match.strings:
                    offset, identifier, data = s
                    print(f" - Offset: {offset}, Identifier: {identifier}, Value: {data}")
        else:
            print("[-] No matches.")

    except yara.Error as e:
        print("YARA scanning error:", e)
```

---

7. Scanning Memory Buffers

If you extract a memory region (e.g., via `volatility`, `psutil`, or from a dump), you can scan directly:

```
def scan_memory_buffer(data, compiled_rules):
    """
    Scan raw memory bytes (e.g., from a dump or process).
    """
    return compiled_rules.match(data=data)
```

🔍 Use Case:

This is ideal when you've already extracted a memory segment from a process or dump and want to scan it inline.

---

8. Organizing Multiple Rules from File System

You can compile many rules from a folder or file:

```
# Compiling rules from a YARA file
compiled = yara.compile(filepath="all_rules.yar")
```

Or even multiple files using a dictionary map:

```
rule_sources = {
    'generic': 'rules/generic.yar',
    'apt': 'rules/apt_rules.yar'
}
compiled = yara.compile(filepaths=rule_sources)
```

---

9. Writing YARA Results to JSON

```
import json

def save_results_to_json(results, output_path="yara_results.json"):
    serializable = []
    for match in results:
        match_dict = {
            "rule": match.rule,
            "meta": match.meta,
            "strings": [
                {"offset": offset, "identifier": ident, "data": str(data)}
                for offset, ident, data in match.strings
            ]
        }
        serializable.append(match_dict)
```

```
    with open(output_path, "w") as f:
        json.dump(serializable, f, indent=2)
```

---

10. Best Practices for YARA Use in Python

- Sanitize user input if loading rule content dynamically.
- Use precompiled `.yar` files for cleaner organization.
- Normalize memory buffers (strip nulls, decode as needed).
- Always handle exceptions (`yara.SyntaxError`, `yara.MatchError`).
- Run rules in isolated containers for sandboxed detection.

---

11. Example Use Case: IOC Matching in Memory Dumps

```
def scan_memory_dump_with_yara(dump_path, rules):
    with open(dump_path, "rb") as f:
        data = f.read()
    return scan_memory_buffer(data, rules)
```

You can extend this logic to automatically scan multiple dumps or files in batch jobs.

---

12. Conclusion

Python and YARA together allow powerful pattern-based scanning of:

- Memory segments

- Executables
- Shellcode dumps
- System snapshots

With `yara-python`, you can:

- Load and apply rules dynamically
- Search memory buffers and disk files
- Automate triage and IOC correlation in real-time